

文档 Agent + AI 短视频创作平台

个人全栈项目 · 2026.07 — 至今

- React Native 动态分包
- FastAPI
- 通义千问
- RAG / Tool Calling
- SSE
- Remotion
- FFmpeg
- CosyVoice
- CDN 热更新

一、简历可用项目介绍（建议直接粘贴）

一句话：在原生动态分包 App 上，先做可溯源的私有文档问答 Agent，再同底座扩展「一句话→分镜→服务端成片→App 内播」的 AI 短视频闭环；渲染按 FFmpeg 兜底 → Remotion 主路径 → Lambda 弹性卸载分层演进。

1.1 简历正文（约 7 条，HR / 技术一眼抓重点）

- 架构：RN 动态分包（问答 docs-agent / 创作 home）+ 独立 FastAPI；问答与视频路由隔离，共用 SSE、取消、评测习惯，避免重渲染拖垮问答。
- 文档 Agent：解析状态机 → Embedding → FAISS（按 KB）→ 手写 Tool Loop；空检索改写、指代改写、Rerank；SSE 工具轨迹 + 可点引用；停止可打断 LLM HTTP。
- 短视频闭环：一句话流式分镜 → TTS 口播 → 服务端成片 → CDN 免鉴权 → App WebView 内嵌播放 / 分享；配图、Logo、多模板、Remix 版本链。
- 渲染分层：FFmpeg 保演示机稳定出片；Remotion 做模板动效与预览同源；规划 Remotion Lambda 只做「渲这一跳」卸载，Schema/Job 不变。
- 成本意识：Remix 按镜头 contentHash 缓存 clip，只重渲变更镜再 concat；API 与渲染可拆独立 worker，失败可恢复。
- 问答在创作中的作用：知识库约束分镜事实；创作助手 Tool 可规划/改镜/入队，形成「问制度 → 出讲解短视频」闭环。
- 工程与演进：固定评测集回归；CDN 热更发版；短中长期覆盖素材层、草稿箱、作品库、多比例导出与发布对接。

1.2 功能清单（分点）

- 已可演示：文档上传/解析/问答引用；一句话成片；配图+Logo；Remix 局部重渲；草稿筛选/复制/分享/重试；多比例入口；页内播放。
- 短期（规划落地）：分镜素材层、局部重渲、草稿箱产品化。
- 中期：Remotion 主路径、独立渲染队列、助手确认后直接入队；Lambda 作为内存紧时的可选渲染跳板。
- 长期：ownerId 作品库、9:16/1:1/16:9 导出、分享卡对接渠道发布。
- 再往后（可实现可演示）：BGM 曲库按模板绑定；口播多音色；分镜拖拽排序；成片字幕样式预设；「从会话一键出片」入口；失败原因看板；简易 A/B 模板对比页。

1.3 难点 · 特色 · 技术点

类型	内容（实事求是）
难点	演示机内存紧导致 Remotion 不稳 → 必须有 FFmpeg 兜底；WebView 误把 favicon 401 当播放失败；局部重渲要保证音画拼接连贯。

特色	有界 Agent + 强 Schema（可渲染、可评测）；问答与创作同 App 双链路隔离；渲染三层可回退，不绑死单一厂商。
技术点	Tool Calling 循环、SSE、状态机队列 recover、FAISS 增量、clip 缓存 concat、Remotion props 与预览协议、CDN 分包热更。

1.4 问答系统最终起什么作用 / 怎么接入

- **角色：**不是附属聊天窗，而是「事实源 + 创作入口」。制度/产品文档先进知识库，回答可引用；创作时可选 KB，planner 强约束避免瞎编卖点。
 - **接入方式：**同一后端、不同路由；创作助手 Tool (plan_storyboard / refine_scene / submit_render) 复用鉴权与 SSE；KB id 写入 Job/Storyboard 元数据可溯源。
 - **终态体验：**在问答里确认条款 → 助手生成讲解分镜 → 一键入队渲染 → 作品库可回看「依据哪份文档」。
-

二、面试要点（分点 Q&A，有深度有广度）

用法：先答黑体「结论句」，再按追问展开。勿夸大「已上生产多租户 / 已全量 Lambda」。

Q1. 这个项目解决什么问题？和「套个 ChatGPT」有何不同？

结论：做可演示的私有知识 Agent + 可渲染的短视频流水线。

不同点：① 文档状态机未 ready 不可检索；② 手写 Tool Loop 有界，不是无限聊天；③ 分镜是强 Schema，能进 Remotion/FFmpeg；④ 有评测集与 requestId/usage 可观测。

Q2. 为什么客户端用 RN 动态分包、后端独立服务？

结论：UI 热更、密钥与重计算不上端。

宿主只加载分包；Agent/RAG/TTS/渲染在 FastAPI；CDN 发 home/docs-agent，避免为改文案重打 APK。LLM Key 仅服务端。

Q3. 为什么问答和短视频要路由隔离？

结论：保护问答主路径 SLA。

渲染吃 CPU/内存；若共用编排易把制度问答拖垮。`/v1/chat` 与 `/v1/video` 分状态机、分队列约定，但复用 SSE/取消/Token 习惯，降低心智成本。

Q4. RAG 怎么做的？幻觉怎么控？

结论：检索强制 + 引用对齐 + 拒答。

切分→Embedding→FAISS（按 KB）→可选 Rerank；模型只能依据 tool 结果；citations 映射合法 `[n]`；无命中拒答。历史窗口截断控 Token。

Q5. Tool Calling 自己写循环还是框架？

结论：服务端手写 loop，便于流式与取消。

每轮：调模型 → 解析 tool_calls → 执行 → 回填 → 再调；SSE 先发 tool 事件再发 delta；cancel 关进行中 HTTP。流式/非流式共用同一 loop。

Q6. FFmpeg / Remotion / Lambda 怎么选？

结论：三层演进，可回退。

FFmpeg：演示机 2G 最稳，字幕/贴图/overlay/concat，保证「一定能出片」。

Remotion：模板动效与预览同源 props，观感主路径。

Lambda：规划为渲染卸载层——同机 Remotion 抢内存或峰值时，只换执行器，不改 Storyboard/Job 协议。当前演示以 auto（Remotion 优先、失败回退 FFmpeg）+ 独立 worker 为主，不把业务绑死 Lambda。

Q7. 为什么不一开始就上 Lambda？

结论：先闭环再弹性，避免过早分布式复杂度。

先验证 Schema、TTS、队列、评测；Lambda 引入鉴权、冷启动、调试成本。等「本机 Remotion 成为瓶颈」再卸，投入产出更清晰。

Q8. Remix 局部重渲怎么做？

结论：contentHash 缓存 clip + 变更镜重渲 + ffmpeg concat。

字段/TTS 文案变则 scene 受影响；未改镜复用旧 clip；保留 parentJobId/版本号。收益：改一镜不必整片重跑，省时省钱。

Q9. 问答最终在短视频里起什么作用？

结论：事实源 + 创作漏斗。

KB 约束卖点/制度表述；助手在确认 patch 后可 create_job/remix 入队；成片可回溯文档依据。差异化是「有据可查的讲解视频」，不是随机营销片。

Q10. 任务队列与失败恢复？

结论：SQLite 持久化状态 + worker 串行/可外置 + 启动 recover。

文档解析与视频渲染同思路：进程挂了能捞回 pending；中期 API 只 enqueue、渲染独立进程，避免渲染堵死 HTTP。

Q11. App 内播放踩过什么坑？

结论：别把子资源 HttpError 当整页失败。

favicon 401 曾误杀 WebView；改为优先加载 `player?embed=1`、软化 onHttpError、媒体失败可重试；成片路径免鉴权供播放器消费。

Q12. 和真正生产还差什么？（加分题）

结论：单机 FAISS/SQLite、进程内队列非分布式、无完整多租户 ACL、Lambda 为架构预留未作为日常默认。

演进：Redis/RQ、对象存储、混合检索、灰度发布、成本看板。能说清差距说明做过工程判断。

Q13. 你怎么证明效果？

结论：双评测 + 真机演示。

文档侧固定语料题集（含 followup）；视频侧分镜 Schema 校验脚本；演示路径：上传→问答引用→一句话成片→Remix→分享。

Q14. 后续还能做什么且能演示？

结论：只承诺可落地的。

BGM 模板绑定、多音色、分镜拖拽、会话「一键出片」、失败原因页、模板 A/B 对比。不做空口「自研大模型视频生成」。

三、可背诵 / 可演讲完整稿（约 3~4 分钟）

【开场 · 30 秒】

面试官你好。我做的是一套「文档 Agent + AI 短视频」个人项目。前端是 React Native 动态分包 App，后端是 FastAPI。先把私有文档问答做稳，再在同一底座上做短视频创作。目标不是堆聊天框，而是把 Agent 工程链路和可演示成片都跑通。

【架构思想 · 40 秒】

架构上我做了两件关键决策：第一，客户端只做体验，密钥、检索、TTS、渲染全在服务端；第二，问答和创作路由隔离——问答走 docs-agent 与 /v1/chat，创作走 home 与 /v1/video——复用 SSE、取消和评测习惯，但状态机分开，避免重渲染拖垮制度问答。文档侧是解析状态机、Embedding、按知识库隔离的 FAISS，加上手写 Tool Calling；回答带可点引用，无证据就拒答。

【短视频与渲染选型 · 60 秒】

短视频链路是：一句话流式生成结构化分镜，CosyVoice 口播，服务端出 MP4，CDN 免鉴权给播放器，App 里 WebView 内嵌播放和分享。渲染我刻意做成三层：FFmpeg 做可靠兜底，保证两 G 内存的演示机也能稳定出片；Remotion 做观感主路径，模板动效和预览用同一套 props；Remotion Lambda 规划成弹性卸载层——当本机 Remotion 抢内存或流量上来时，只换「渲这一跳」的执行器，Schema 和 Job 协议不变。这样不会一上来就被云渲染绑死，也能讲清什么时候该上 Lambda。

【问答如何接入创作 · 40 秒】

问答不是旁路。知识库是事实源：创作可选 KB，planner 强约束文案；创作助手在确认修改后可以直接入队渲染。终态体验是：在问答里核对条款，一键生成讲解短视频，作品还能回溯依据哪份文档。另外 Remix 按镜头做 clip 缓存，只重渲变更镜再拼接，这是成本和体验上的工程点。

【演进与边界 · 40 秒】

短中长期我规划了素材与 Logo、局部重渲、草稿箱、独立渲染 worker、作品库、多比例和发布对接；再往后可以做 BGM、多音色、分镜拖拽、会话一键出片等可演示能力。我也清楚差距：现在是单机 FAISS 和 SQLite，队列可外置但不是完整分布式，Lambda 是架构预留。今天可以按您关心的方向展开：检索质量、Agent 边界，或渲染分层与局部重渲。

30 秒电梯版（HR / 初筛）

我做了文档问答 Agent 加 AI 短视频创作。RN 分包 App + FastAPI；问答可引用私有文档；一句话能生成分镜并服务端成片。渲染用 FFmpeg 保底、Remotion 提观感，并规划 Lambda 做弹性卸载。问答知识库还能约束短视频内容，形成「有据讲解视频」。有评测和真机演示。

自我介绍钩子（可接追问）

- 想听检索 / 幻觉 → 引 Q4、Q5
- 想听系统设计 → 引 Q2、Q3、Q10
- 想听多媒体 / 渲染 → 引 Q6、Q7、Q8
- 想听产品闭环 → 引 Q9、Q14

材料说明：与仓库 docs-agent-server / rn-biz-0.86 / rn-dynamic-0.86 现状一致；「Lambda」表述为架构演进与规划接入，面试时勿说成「线上已默认全量 Lambda」。评测数字以当时 eval 结果为准，口述可用「固定题集回归」。